UNIT 4

Turing Machine

Turing machine was invented in 1936 by Alan Turing. It is an accepting device which accepts Recursive Enumerable Language generated by type 0 grammar.

There are various features of the Turing machine:

1. **It has an external memory which remembers arbitrary long sequence of input.**
2. **It has unlimited memory capability.**
3. **The model has a facility by which the input at left or right on the tape can be read easily.**
4. **The machine can produce a certain output based on its input. Sometimes it may be required that the same input has to be used to generate the output. So in this machine, the distinction between input and output has been removed. Thus a common set of alphabets can be used for the Turing machine.**

Formal definition of Turing machine

A Turing machine can be defined as a collection of 7 components:

Q: the finite set of states
∑: the finite set of input symbols
T: the tape symbol
q0: the initial state
F: a set of final states
B: a blank symbol used as a end marker for input
δ: a transition or mapping function.

The mapping function shows the mapping from states of finite automata and input symbol on the tape to the next states, external symbols and the direction for moving the tape head. This is known as a triple or a program for turing machine.

1. **(q0, a) → (q1, A, R)**

   That means in q0 state, if we read symbol 'a' then it will go to state q1, replaced a by X and move ahead right(R stands for right).

Example:

Construct TM for the language L ={0^n1^n} where n>=1.

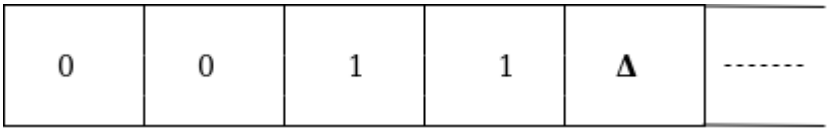
Solution:

We have already solved this problem by PDA. In PDA, we have a stack to remember the previous symbol. The main advantage of the Turing machine is we have a tape head which can be moved forward or backward, and the input tape can be scanned.

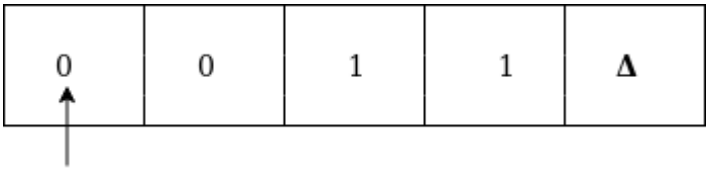
The simple logic which we will apply is read out each '0' mark it by A and then move ahead along with the input tape and find out 1 convert it to B. Now, repeat this process for all a's and b's.

Now we will see how this turing machine work for 0011.

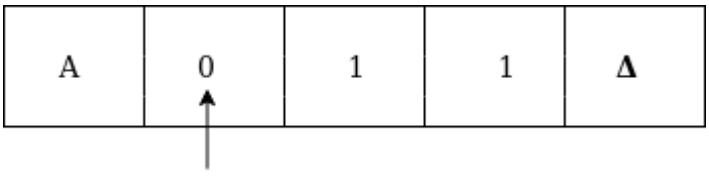
The simulation for 0011 can be shown as below:



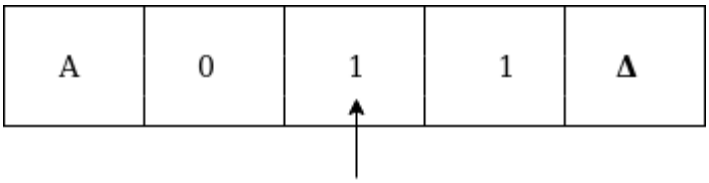
Now, we will see how this turing machine will works for 0011. Initially, state is q0 and head points to 0 as:



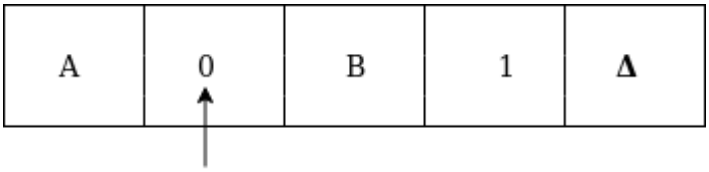
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A and head will move to the right as:



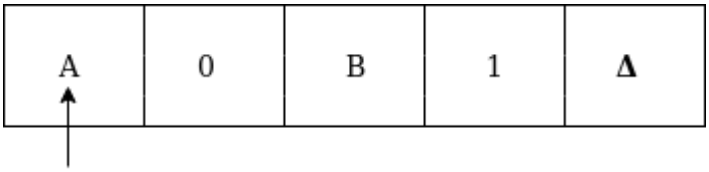
The move will be δ(q1, 0) = δ(q1, 0, R) which means it will not change any symbol, remain in the same state and move to the right as:



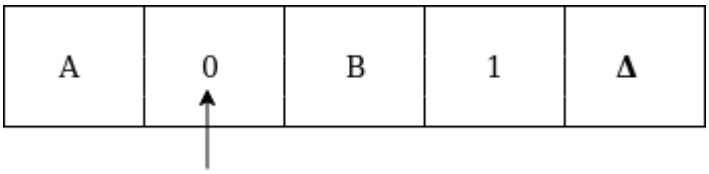
The move will be δ(q1, 1) = δ(q2, B, L) which means it will go to state q2, replaced 1 by B and head will move to left as:



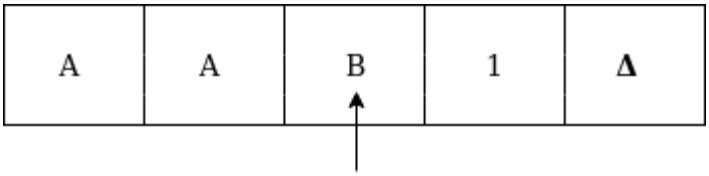
Now move will be δ(q2, 0) = δ(q2, 0, L) which means it will not change any symbol, remain in the same state and move to left as:



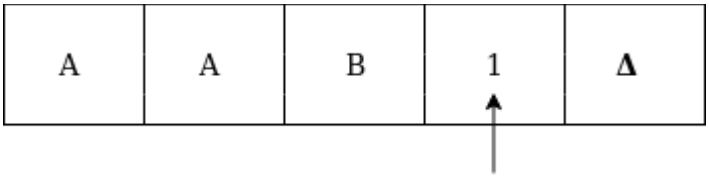
The move will be δ(q2, A) = δ(q0, A, R), it means will go to state q0, replaced A by A and head will move to the right as:



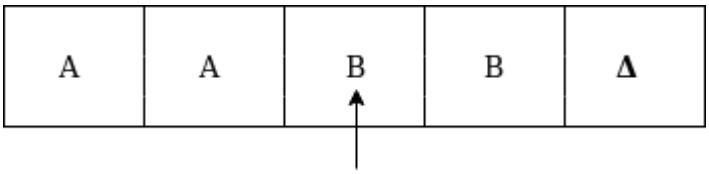
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A, and head will move to right as:



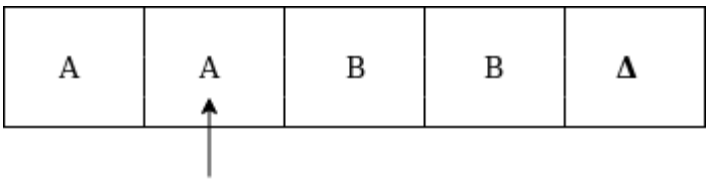
The move will be δ(q1, B) = δ(q1, B, R) which means it will not change any symbol, remain in the same state and move to right as:



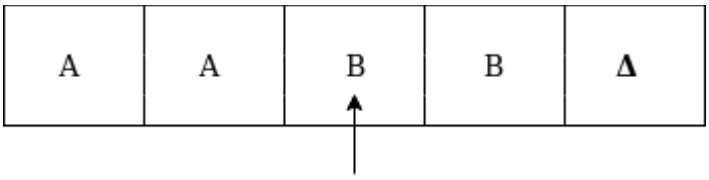
The move will be δ(q1, 1) = δ(q2, B, L) which means it will go to state q2, replaced 1 by B and head will move to left as:



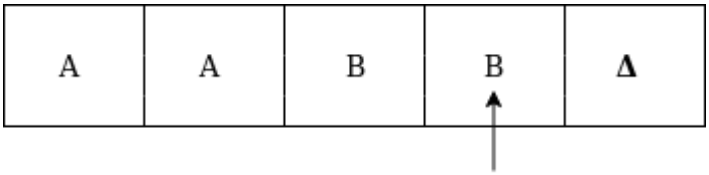
The move δ(q2, B) = (q2, B, L) which means it will not change any symbol, remain in the same state and move to left as:



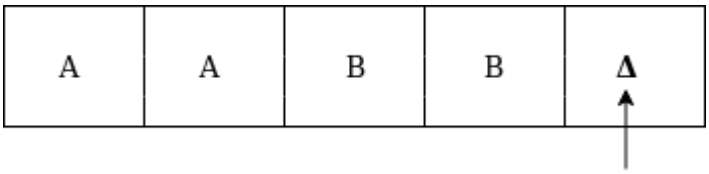
Now immediately before B is A that means all the 0?s are market by A. So we will move right to ensure that no 1 is present. The move will be δ(q2, A) = (q0, A, R) which means it will go to state q0, will not change any symbol, and move to right as:



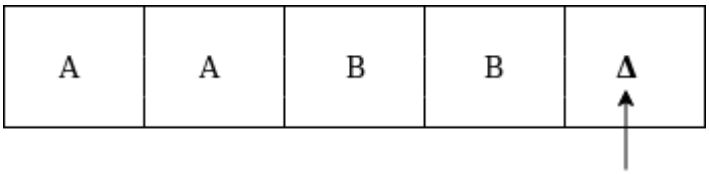
The move δ(q0, B) = (q3, B, R) which means it will go to state q3, will not change any symbol, and move to right as:



The move δ(q3, B) = (q3, B, R) which means it will not change any symbol, remain in the same state and move to right as:



The move δ(q3, Δ) = (q4, Δ, R) which means it will go to state q4 which is the HALT state and HALT state is always an accept state for any TM.

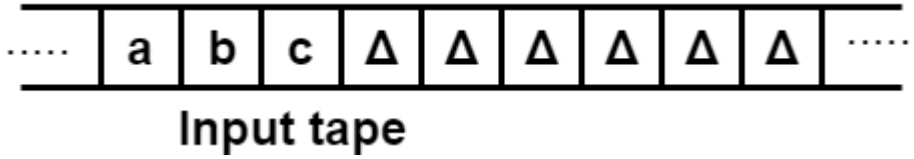


The same TM can be represented by Transition Diagram:

Basic Model of Turing machine

The turning machine can be modelled with the help of the following representation.

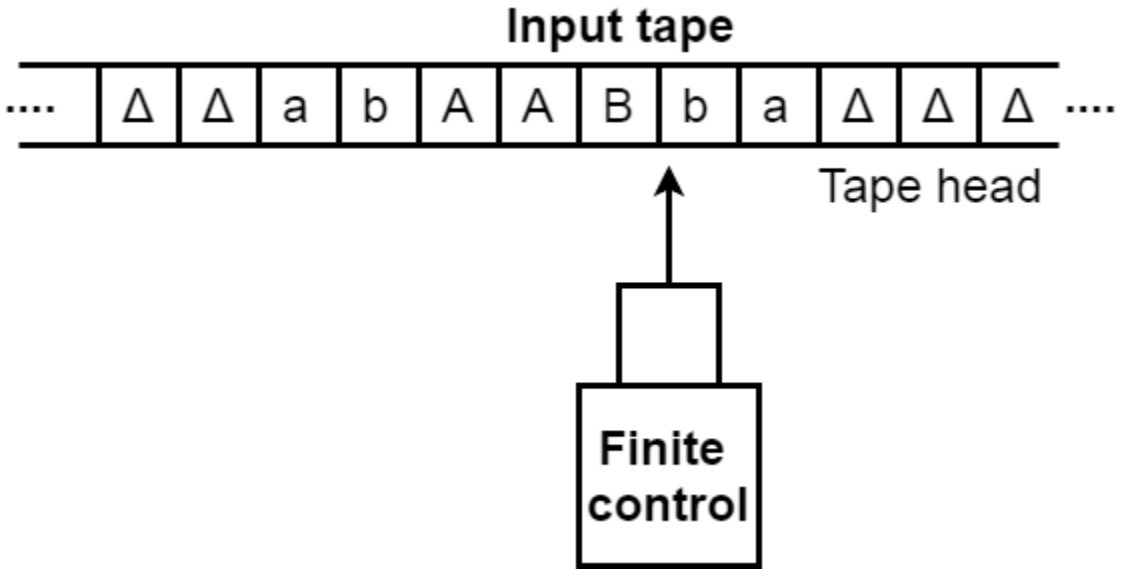
1. The input tape is having an infinite number of cells, each cell containing one input symbol and thus the input string can be placed on tape. The empty tape is filled by blank characters.



2. The finite control and the tape head which is responsible for reading the current input symbol. The tape head can move to left to right.

3. A finite set of states through which machine has to undergo.

4. Finite set of symbols called external symbols which are used in building the logic of turing machine.



Language accepted by Turing machine

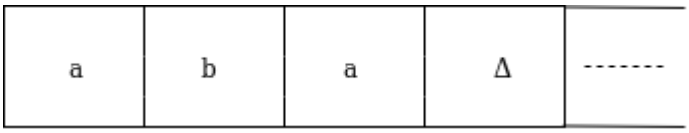
The turing machine accepts all the language even though they are recursively enumerable. Recursive means repeating the same set of rules for any number of times and enumerable means a list of elements. The TM also accepts the computable functions, such as addition, multiplication, subtraction, division, power function, and many more.

Example:

Construct a turing machine which accepts the language of aba over ∑ = {a, b}.

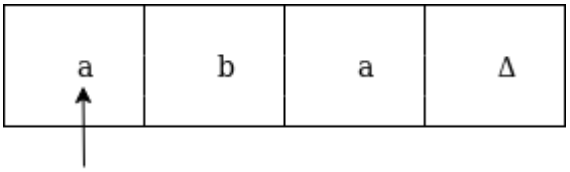
Solution:

We will assume that on input tape the string 'aba' is placed like this:

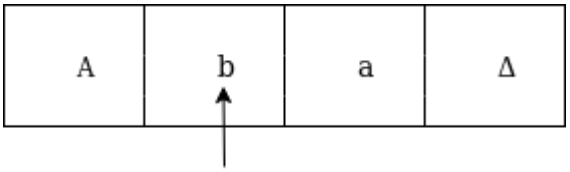


The tape head will read out the sequence up to the Δ characters. If the tape head is readout 'aba' string then TM will halt after reading Δ.

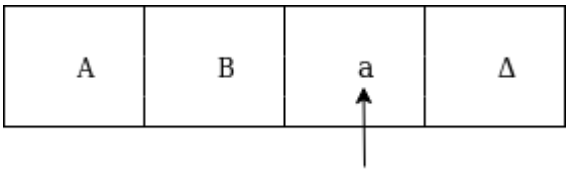
Now, we will see how this turing machine will work for aba. Initially, state is q0 and head points to a as:



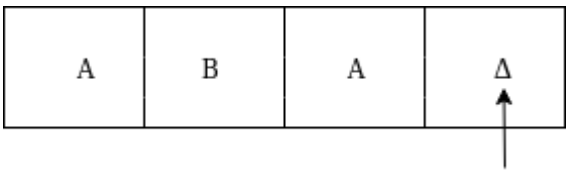
The move will be δ(q0, a) = δ(q1, A, R) which means it will go to state q1, replaced a by A and head will move to right as:



The move will be δ(q1, b) = δ(q2, B, R) which means it will go to state q2, replaced b by B and head will move to right as:



The move will be δ(q2, a) = δ(q3, A, R) which means it will go to state q3, replaced a by A and head will move to right as:

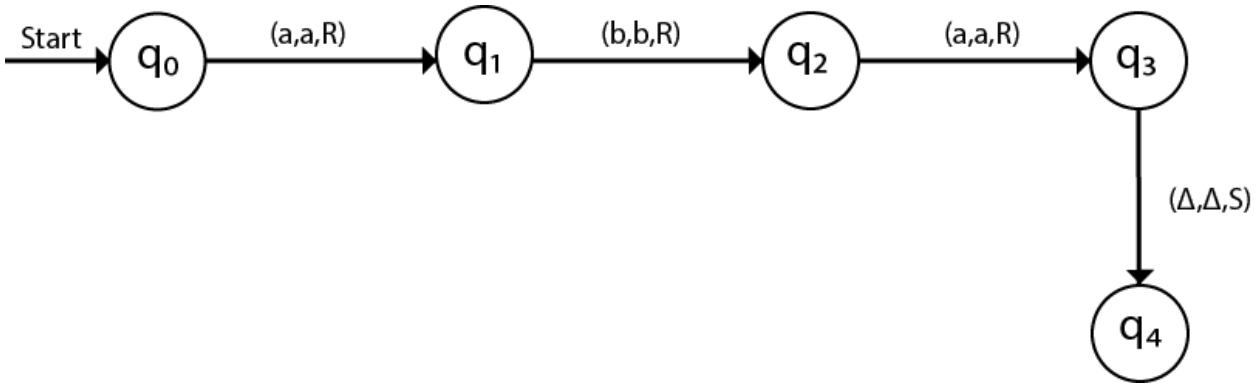


The move δ(q3, Δ) = (q4, Δ, S) which means it will go to state q4 which is the HALT state and HALT state is always an accept state for any TM.

The same TM can be represented by Transition Table:

States	a	b	Δ
q0	(q1, A, R)	–	–
q1	–	(q2, B, R)	–
q2	(q3, A, R)	–	–
q3	–	–	(q4, Δ, S)
q4	–	–	–

The same TM can be represented by Transition Diagram:



Examples of TM

Example 1:

DurationÂ 18:10

Loaded: 0.37%

Â

Fullscreen

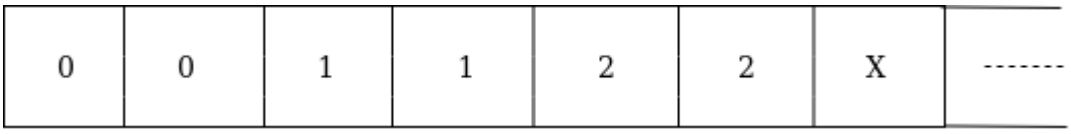
Backward Skip 10sPlay VideoForward Skip 10s

Construct a TM for the language L = {$0^n1^n2^n$} where n≥1

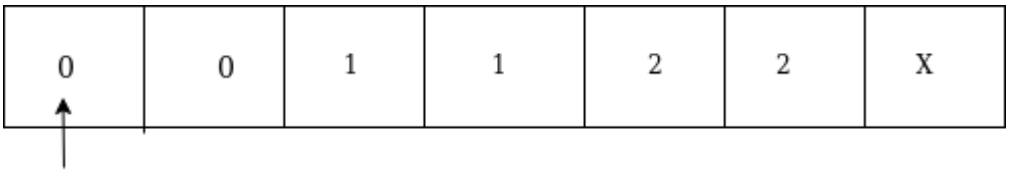
Solution:

L = {$0^n1^n2^n$ | n≥1} represents language where we use only 3 character, i.e., 0, 1 and 2. In this, some number of 0's followed by an equal number of 1's and then followed by an equal number of 2's. Any type of string which falls in this category will be accepted by this language.

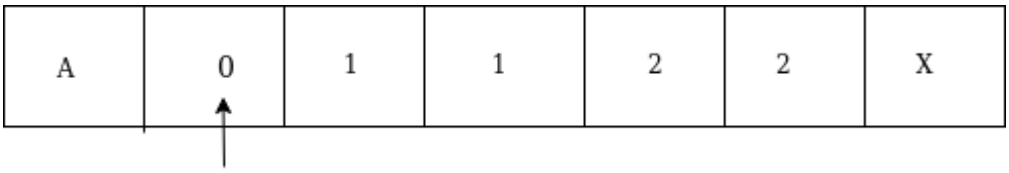
The simulation for 001122 can be shown as below:



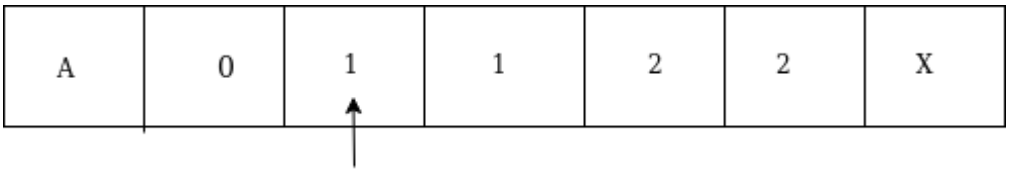
Now, we will see how this Turing machine will work for 001122. Initially, state is q0 and head points to 0 as:



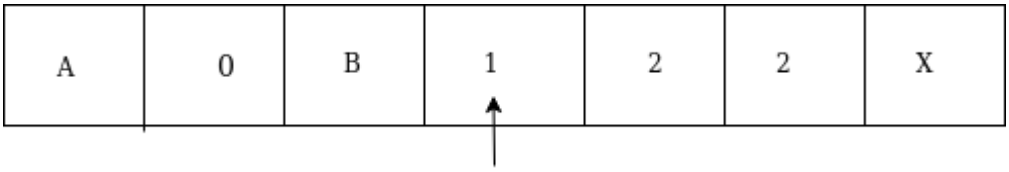
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A and head will move to the right as:



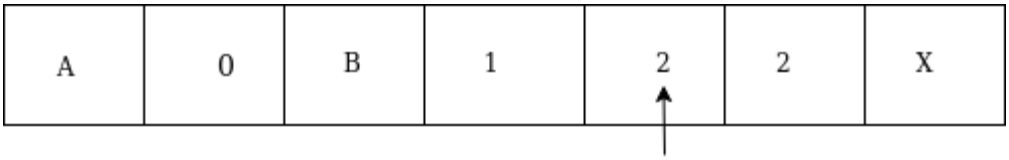
The move will be δ(q1, 0) = δ(q1, 0, R) which means it will not change any symbol, remain in the same state and move to the right as:



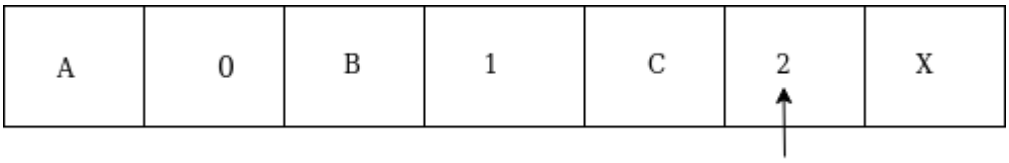
The move will be δ(q1, 1) = δ(q2, B, R) which means it will go to state q2, replaced 1 by B and head will move to right as:



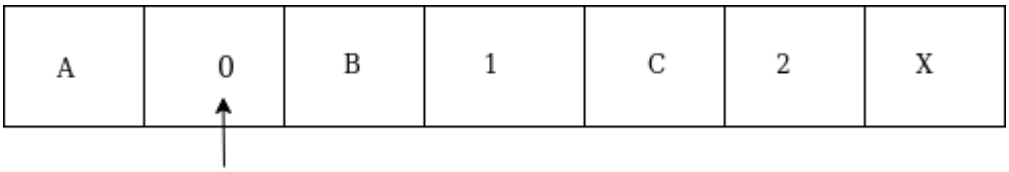
The move will be δ(q2, 1) = δ(q2, 1, R) which means it will not change any symbol, remain in the same state and move to right as:



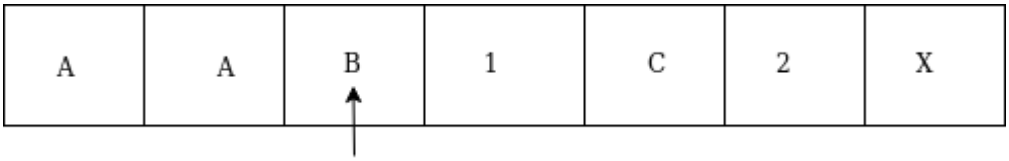
The move will be δ(q2, 2) = δ(q3, C, R) which means it will go to state q3, replaced 2 by C and head will move to right as:



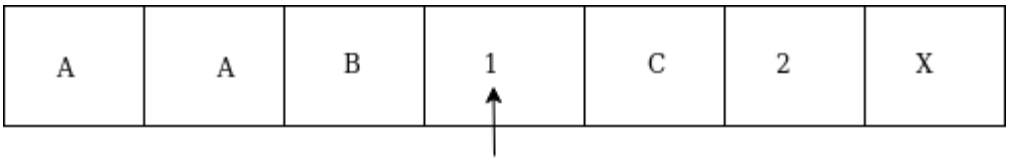
Now move δ(q3, 2) = δ(q3, 2, L) and δ(q3, C) = δ(q3, C, L) and δ(q3, 1) = δ(q3, 1, L) and δ(q3, B) = δ(q3, B, L) and δ(q3, 0) = δ(q3, 0, L), and then move δ(q3, A) = δ(q0, A, R), it means will go to state q0, replaced A by A and head will move to right as:



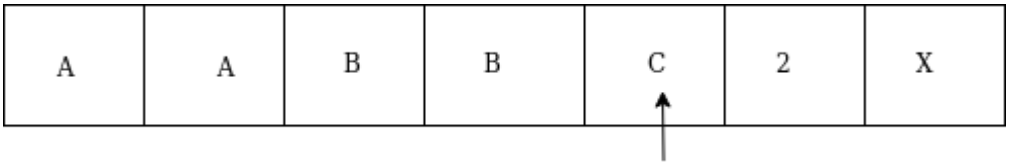
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A, and head will move to right as:



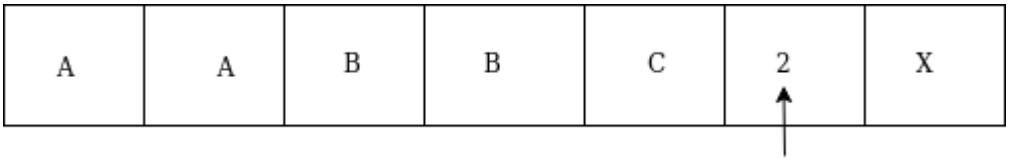
The move will be δ(q1, B) = δ(q1, B, R) which means it will not change any symbol, remain in the same state and move to right as:



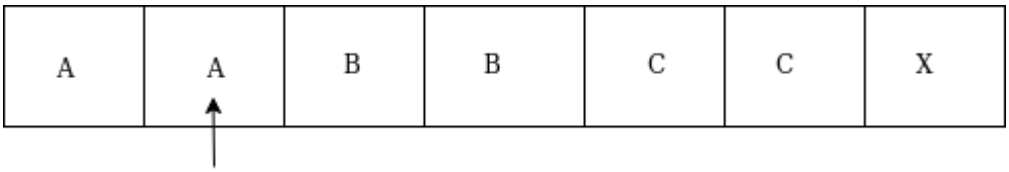
The move will be δ(q1, 1) = δ(q2, B, R) which means it will go to state q2, replaced 1 by B and head will move to right as:



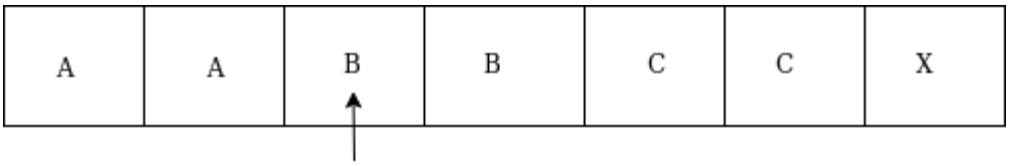
The move will be δ(q2, C) = δ(q2, C, R) which means it will not change any symbol, remain in the same state and move to right as:



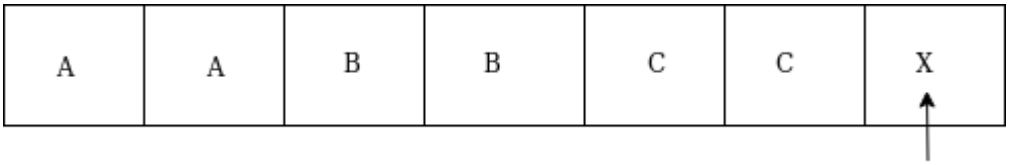
The move will be δ(q2, 2) = δ(q3, C, L) which means it will go to state q3, replaced 2 by C and head will move to left until we reached A as:



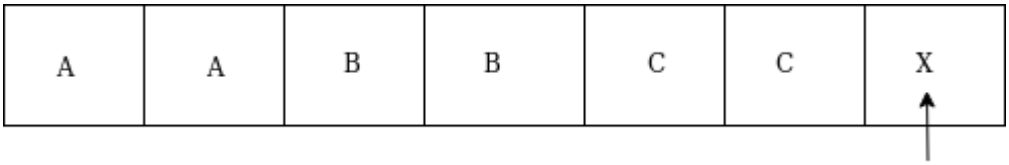
immediately before B is A that means all the 0's are market by A. So we will move right to ensure that no 1 or 2 is present. The move will be δ(q2, B) = (q4, B, R) which means it will go to state q4, will not change any symbol, and move to right as:



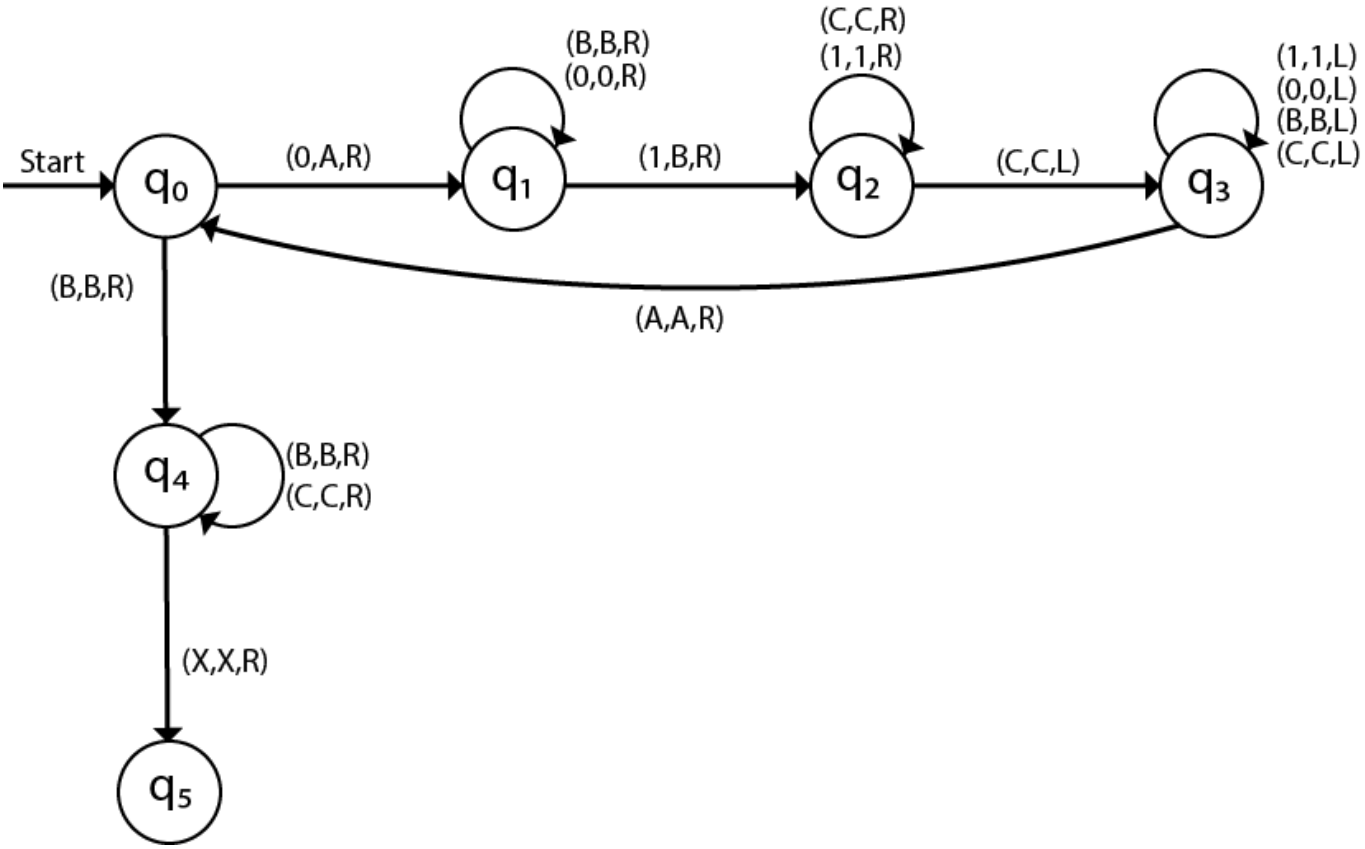
The move will be (q4, B) = δ(q4, B, R) and (q4, C) = δ(q4, C, R) which means it will not change any symbol, remain in the same state and move to right as:



The move δ(q4, X) = (q5, X, R) which means it will go to state q5 which is the HALT state and HALT state is always an accept state for any TM.



The same TM can be represented by Transition Diagram:



Example 2:

Construct a TM machine for checking the palindrome of the string of even length.

Solution:

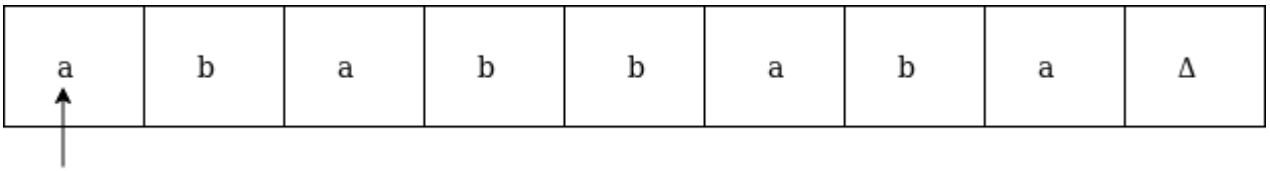
Firstly we read the first symbol from the left and then we compare it with the first symbol from right to check whether it is the same.

Again we compare the second symbol from left with the second symbol from right. We repeat this process for all the symbols. If we found any symbol not matching, we cannot lead the machine to HALT state.

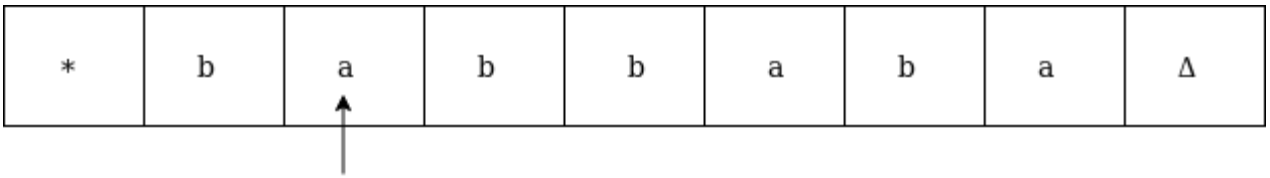
Suppose the string is ababbabaΔ. The simulation for ababbabaΔ can be shown as follows:

a	b	a	b	b	a	b	a	Δ	---------

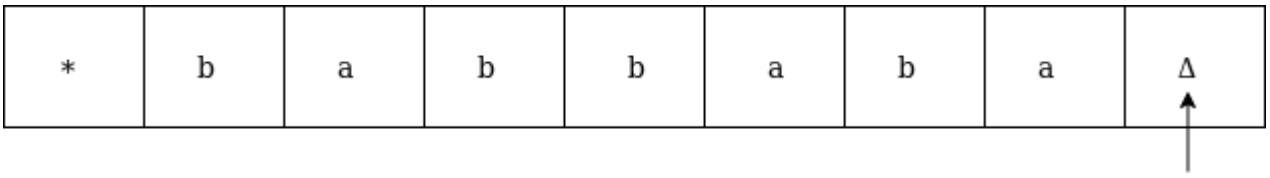
Now, we will see how this Turing machine will work for ababbabaΔ. Initially, state is q0 and head points to a as:



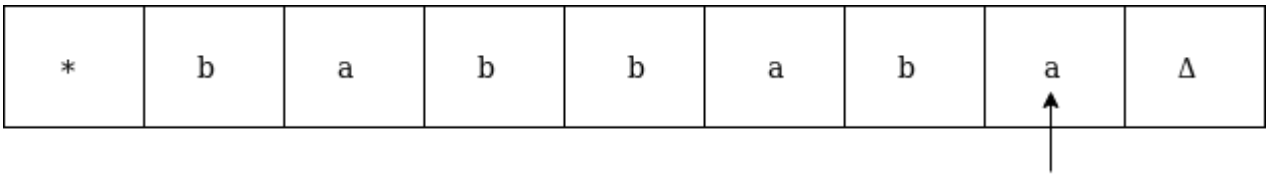
We will mark it by * and move to right end in search of a as:



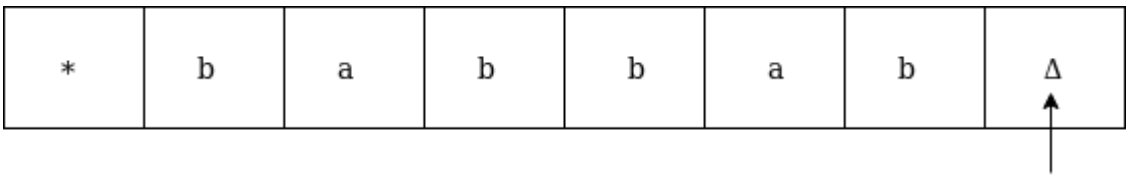
We will move right up to Δ as:



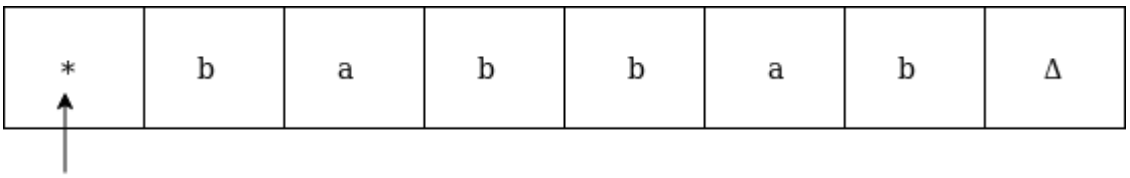
We will move left and check if it is a:



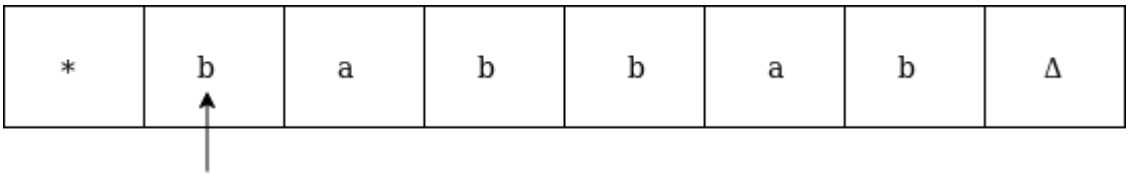
It is 'a' so replace it by Δ and move left as:



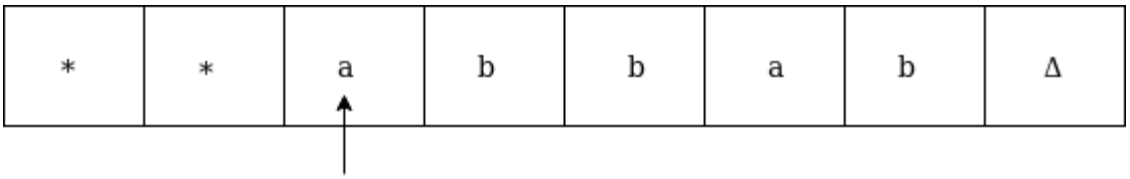
Now move to left up to * as:



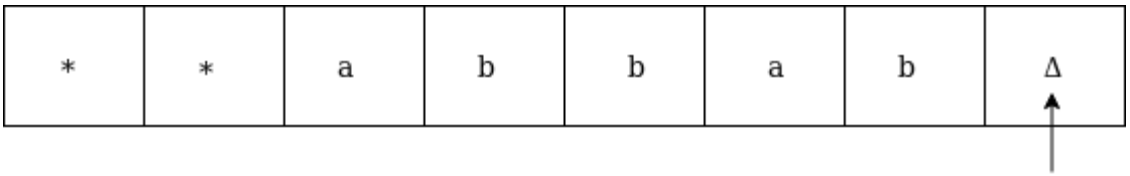
Move right and read it



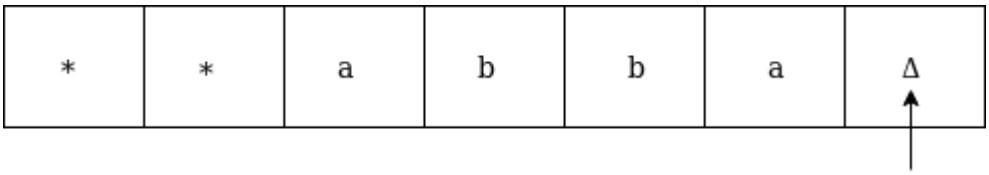
Now convert b by * and move right as:



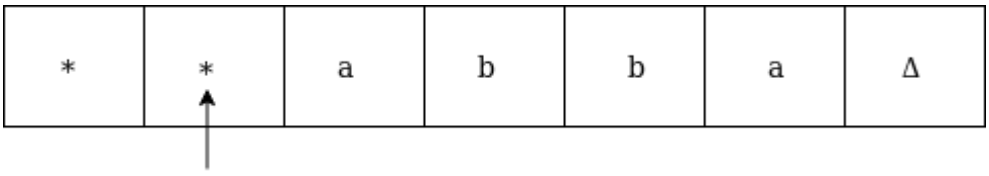
Move right up to Δ in search of b as:



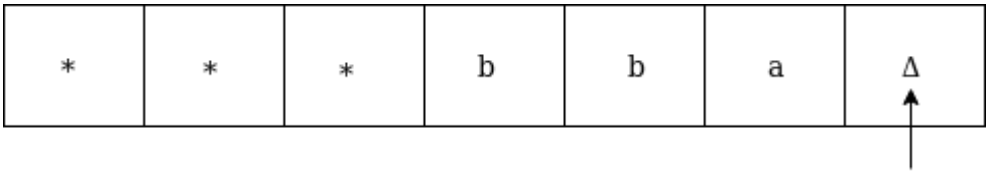
Move left, if the symbol is b then convert it into Δ as:



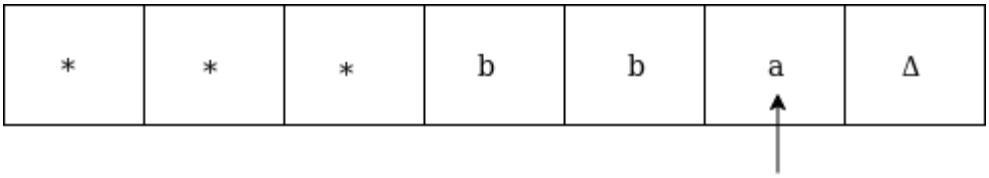
Now move left until * as:



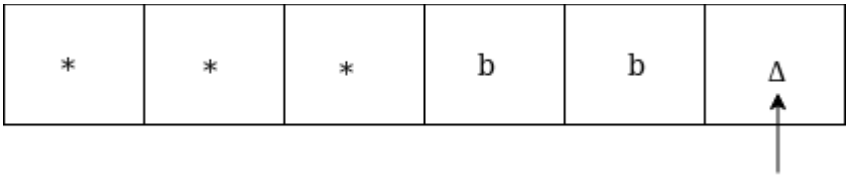
Replace a by * and move right up to Δ as:



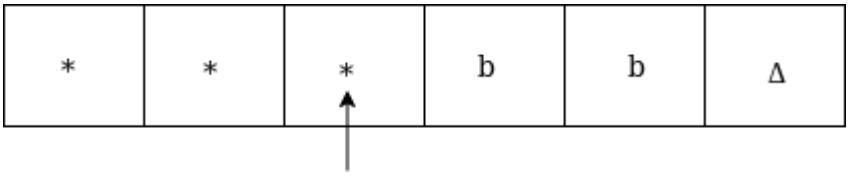
We will move left and check if it is a, then replace it by Δ as:



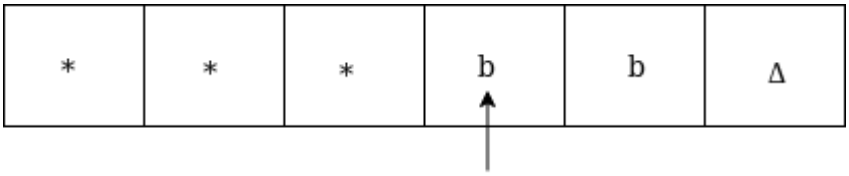
It is 'a' so replace it by Δ as:



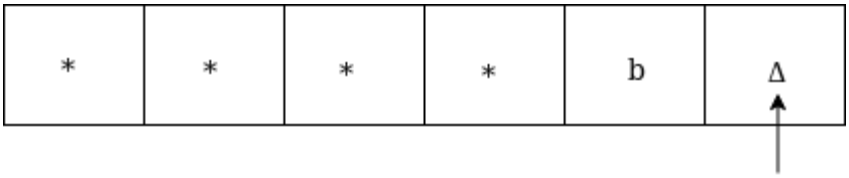
Now move left until *



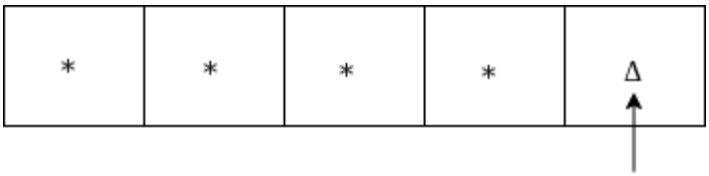
Now move right as:



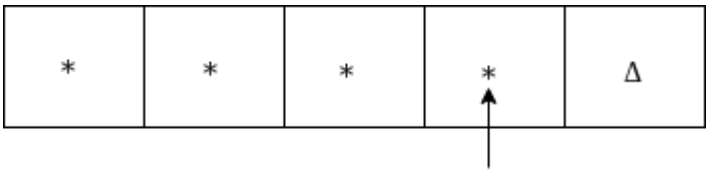
Replace b by * and move right up to Δ as:



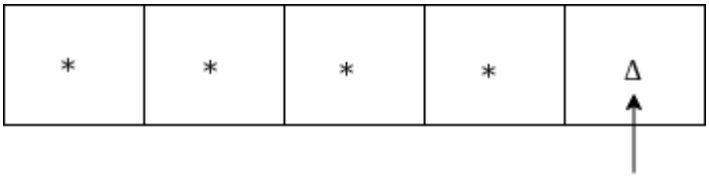
Move left, if the left symbol is b, replace it by Δ as:



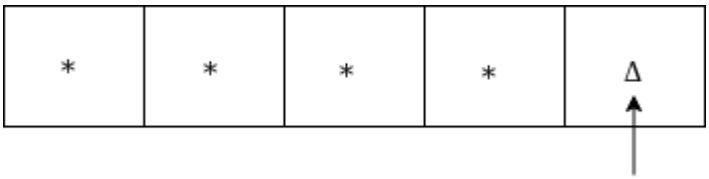
Move left till *



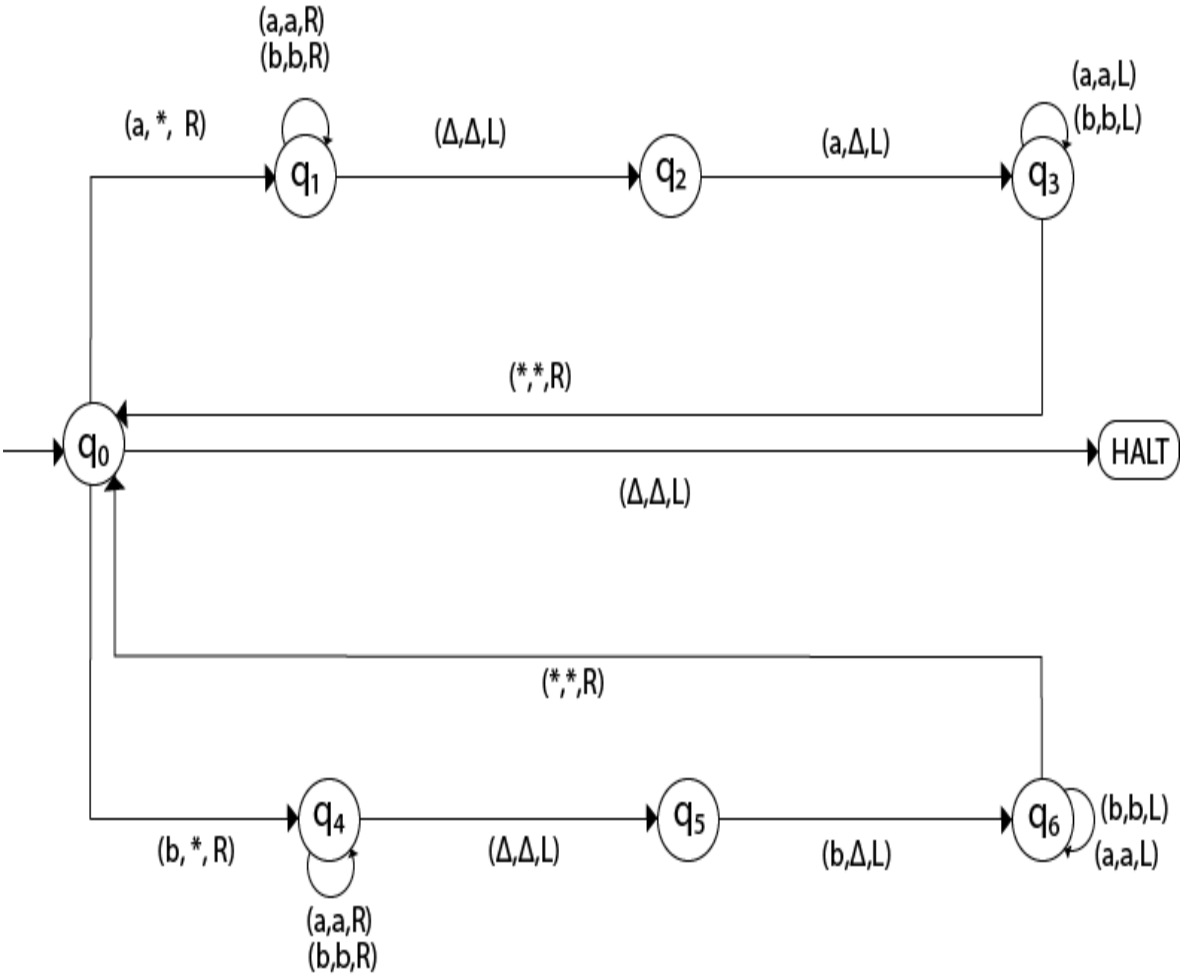
Move right and check whether it is Δ



Go to HALT state



The same TM can be represented by Transition Diagram:



Introduction to Undecidability

In the theory of computation, we often come across such problems that are answered either 'yes' or 'no'. The class of problems which can be answered as 'yes' are called solvable or decidable. Otherwise, the class of problems is said to be unsolvable or undecidable.

Undecidability of Universal Languages:

The universal language L_u is a recursively enumerable language and we have to prove that it is undecidable (non-recursive).

Theorem: L_u is RE but not recursive.

Proof:

Consider that language L_u is recursively enumerable language. We will assume that L_u is recursive. Then the complement of L_u that is L`u is also recursive. However, if we have a TM M to accept L`u then we can construct a TM L_d. But L_d the diagonalization language is

not RE. Thus our assumption that L_u is recursive is wrong (not RE means not recursive). Hence we can say that L_u is RE but not recursive. The construction of M for L_d is as shown in the following diagram:

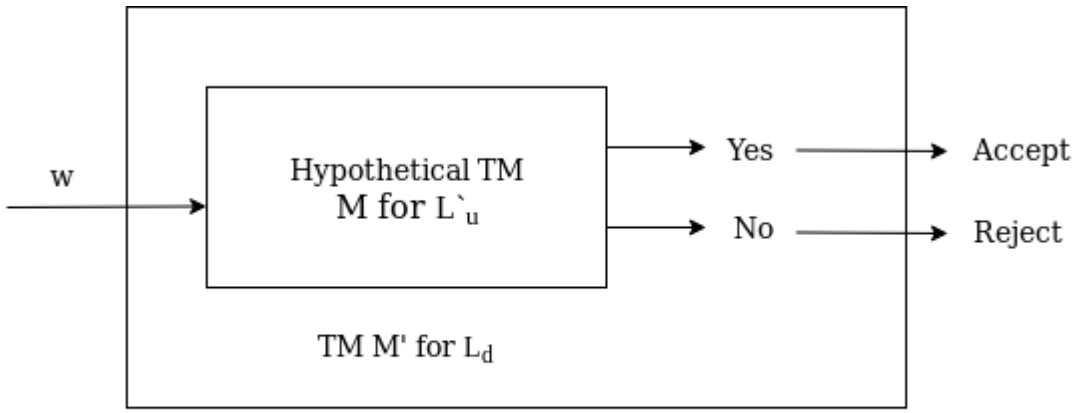


Fig: Construction of M' for L_d

Chomsky Hierarchy

Chomsky Hierarchy represents the class of languages that are accepted by the different machine. The category of language in Chomsky's Hierarchy is as given below:

1. **Type 0 known as Unrestricted Grammar.**
2. **Type 1 known as Context Sensitive Grammar.**
3. **Type 2 known as Context Free Grammar.**
4. **Type 3 Regular Grammar.**

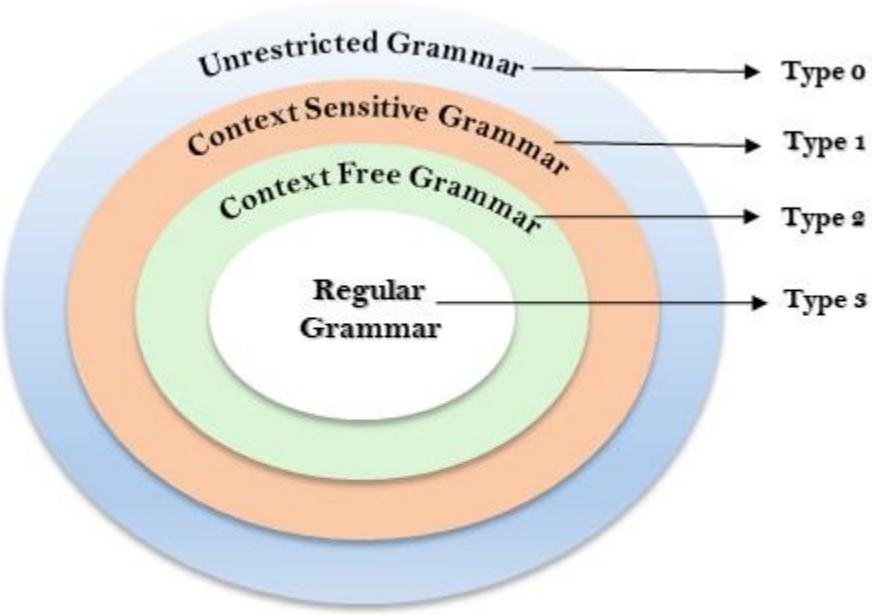


Fig: Chomsky Hierarchy

This is a hierarchy. Therefore every language of type 3 is also of type 2, 1 and 0. Similarly, every language of type 2 is also of type 1 and type 0, etc.

Type 0 Grammar:

Type 0 grammar is known as Unrestricted grammar. There is no restriction on the grammar rules of these types of languages. These languages can be efficiently modeled by Turing machines.

For example:

1. **bAa → aa**
2. **S → s**

Type 1 Grammar:

Type 1 grammar is known as Context Sensitive Grammar. The context sensitive grammar is used to represent context sensitive language. The context sensitive grammar follows the following rules:

- **The context sensitive grammar may have more than one symbol on the left hand side of their production rules.**
- **The number of symbols on the left-hand side must not exceed the number of symbols on the right-hand side.**

- **The rule of the form A → ε is not allowed unless A is a start symbol. It does not occur on the right-hand side of any rule.**
- **The Type 1 grammar should be Type 0. In type 1, Production is in the form of V → T**

Where the count of symbol in V is less than or equal to T.

For example:

1. **S → AT**
2. **T → xy**
3. **A → a**

Type 2 Grammar:

Type 2 Grammar is known as Context Free Grammar. Context free languages are the languages which can be represented by the context free grammar (CFG). Type 2 should be type 1. The production rule is of the form

1. **A → α**

Where A is any single non-terminal and is any combination of terminals and non-terminals.

For example:

1. **A → aBb**
2. **A → b**
3. **B → a**

Type 3 Grammar:

Type 3 Grammar is known as Regular Grammar. Regular languages are those languages which can be described using regular expressions. These languages can be modeled by NFA or DFA.

Type 3 is most restricted form of grammar. The Type 3 grammar should be Type 2 and Type 1. Type 3 should be in the form of

1. **V → T*V / T***

For example:

1. **$A \rightarrow xy$**